

DEVELOPMENT



HelixCode Development Guide

Getting Started

Prerequisites

- Go 1.26+ (this inner module declares `go 1.26` — it will not build on older toolchains)
- PostgreSQL 15+
- Redis 7+
- Git

Development Setup

1. **Clone the repository:**

```
git clone <repository-url>
cd HelixCode
```

2. **Setup dependencies:**

```
make setup-deps
```

3. **Generate logo assets:**

```
make logo-assets
```

4. **Build the application:**

```
make build
```

5. **Run development server:**

```
make dev
```

Project Architecture

Core Components

1. Database Layer (**internal/database/**)

- **database.go**: PostgreSQL connection pool and schema management
- **Features**: Connection pooling, health checks, automatic schema creation

2. Authentication (**internal/auth/**)

- **auth.go**: JWT-based authentication with session management
- **Features**: User registration, login, token refresh, password hashing

3. Worker Management (**internal/worker/**)

- **manager.go**: Distributed worker registration and health monitoring
- **Features**: Worker discovery, capability-based assignment, health checks

4. Task Management (**internal/task/**)

- **manager.go**: Distributed task management with work preservation
- **Features**: Task creation, assignment, checkpointing, rollback

5. Server (**internal/server/**)

- **server.go**: HTTP server with REST API endpoints
- **Features**: Gin framework, middleware, route management

6. Configuration (**internal/config/**)

- **config.go**: Configuration management with Viper
- **Features**: Environment variables, config file support, validation

7. Logo Processing (**internal/logo/**)

- **processor.go**: Logo asset generation and color extraction
- **Features**: ASCII art generation, icon creation, theme generation

Development Workflow

Code Organization

```
internal/  
├── auth/           # Authentication & authorization  
├── config/         # Configuration management  
├── database/       # Database layer  
├── logo/           # Logo processing  
├── server/         # HTTP server  
├── task/           # Task management  
├── theme/          # Color themes  
└── worker/        # Worker management
```

Adding New Features

1. **Define the data model** in the database schema
2. **Create repository interface** for data access
3. **Implement business logic** in the appropriate manager
4. **Add API endpoints** in the server
5. **Write comprehensive tests**
6. **Update documentation**

Testing Strategy

```
# Run all tests  
make test  
  
# Run specific package tests  
go test -v ./internal/auth  
  
# Run tests with coverage  
go test -cover ./...  
  
# Run integration tests  
go test -tags=integration ./...
```

Code Quality

```
# Format code  
make fmt  
  
# Lint code  
make lint
```



```
# Check dependencies
go mod tidy
```

API Development

Adding New Endpoints

1. **Define route** in `internal/server/server.go`
2. **Implement handler method**
3. **Add request/response models**
4. **Update OpenAPI documentation**
5. **Add tests**

Example: Adding User Profile Endpoint

```
// 1. Add route in setupRoutes()
users.GET("/profile", s.getUserProfile)

// 2. Implement handler
func (s *Server) getUserProfile(c *gin.Context) {
    // Implementation
}
```

Database Development

Schema Changes

1. **Update schema** in `internal/database/database.go`
2. **Create migration** if needed
3. **Update repository interfaces**
4. **Update tests**

Adding New Table

```
-- Add to createSchemaSQL in database.go
CREATE TABLE new_table (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    -- columns
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Design System Development

Color Scheme

Colors are automatically extracted from the logo. To update:

1. **Replace logo.png** in `assets/images/`
2. **Run logo asset generation:**

```
make logo-assets
```

Adding New Themes

1. **Update theme generation** in `internal/logo/processor.go`
2. **Add theme constants** in `internal/theme/theme.go`

✦

3. **Regenerate assets:**

```
make logo-assets
```

Integration Development

Adding New LLM Providers

1. **Add provider configuration** in `config`
2. **Implement provider interface**
3. **Add to task capabilities**
4. **Update worker registration**

Adding New Task Types

1. **Define task type** in `internal/task/manager.go`
2. **Add required capabilities**
3. **Implement task execution logic**
4. **Update API endpoints**

Deployment

Development Deployment

```
# Build for development  
make build
```

```
# Run with development config
./bin/helixcode --config config/dev/config.yaml
```

Production Deployment

```
# Build for production
make prod
```

```
# Run with production config
./bin/helixcode --config config/prod/config.yaml
```

Containerized deployment

Per Constitution Rule 4 (No Manual Container Commands) + §11.4.76, container workflows go through the orchestrator — direct `docker/docker-compose` commands are NOT a supported workflow (and the host uses `podman`, not `docker`). Run the platform standalone via the repo-root `./helix` facade script (it drives the containerized stack, delegating to the containers submodule per §11.4.76):

```
./helix start      # boot the containerized stack
./helix status     # show stack status
./helix logs       # tail logs
./helix restart    # restart the stack
./helix stop       # tear down
```

The host application binary itself is built with `make build` → `./bin/helixcode` (see Build Commands above); CI-style container builds/tests are driven by the orchestrator, never by hand-run `docker` commands.

Debugging

Logging

- **Development:** Set `logging.level: debug` in config
- **Production:** Set `logging.level: info` or `warn`

Database Debugging

```
# Connect to database
psql -h localhost -U helixcode -d helixcode

# Check connections
SELECT * FROM pg_stat_activity;
```

API Debugging

Test health endpoint

```
curl http://localhost:8080/health
```

Test API endpoints

```
curl -H "Authorization: Bearer <token>" http://localhost:8080/  
api/v1/users/me
```

Monitoring

Health Checks

- **Database:** GET /health
- **Worker status:** Monitor worker heartbeats
- **Task progress:** Track task completion rates

Metrics

- **Worker metrics:** CPU, memory, disk usage
- **Task metrics:** Completion time, success rates
- **System metrics:** API response times, error rates

Security

Authentication

- Use JWT tokens for API access
- Implement session management
- Use environment variables for secrets

Input Validation

- Validate all user inputs
- Use prepared statements for database queries
- Implement rate limiting

Security Headers

- CORS configuration
- XSS protection
- Content security policy

Performance

Database Optimization

- Use connection pooling
- Implement database indexes
- Monitor query performance

Caching

- Use Redis for session storage
- Implement response caching
- Cache frequently accessed data

Worker Optimization

- Load balancing across workers
- Resource monitoring
- Auto-scaling based on load

Contributing

Code Review Process

1. **Create feature branch** from main
2. **Make changes** with tests
3. **Submit pull request**
4. **Address review comments**
5. **Merge after approval**

Commit Guidelines

- Use conventional commit messages
- Include tests with new features
- Update documentation
- Ensure code passes CI checks

Release Process

1. **Update version** in main.go
2. **Update changelog**
3. **Create release tag**
4. **Build release binaries**
5. **Deploy to production**

Happy coding!

Sources verified

Per constitution §11.4.99, these development instructions were cross-referenced against the latest official sources + the repository's actual state on **2026-05-29**:

- **Go version** — corrected “Go 1.21+” → “**Go 1.26+**” to match `helix_code/go.mod` (`go 1.26`; won't build below it). Verified against <https://go.dev/doc/devel/release> (Go 1.26.0 released 2026-02-10, latest 1.26.3; Go 1.21/1.24 are past support).
- **make targets** — `setup-deps`, `logo-assets`, `build`, `dev`, `test`, `fmt`, `lint`, `prod` all verified present in `helix_code/Makefile`.
- **Container workflow** — removed the `docker build/docker run` instructions (violate Constitution Rule 4 + the host uses `podman`, not `docker`) and replaced them with the real `repo-root ./helix` facade (subcommands verified: `start/status/logs/restart/stop`). Negative finding: the `make container-*` targets referenced in `CLAUDE.md` §3.4 do NOT exist in either `Makefile` — flagged for a separate `CLAUDE.md` correction (not cited here).
- **PostgreSQL 15+ / Redis 7+** — consistent with `CLAUDE.md` §3.1.

Sources verified 2026-05-29: <https://go.dev/doc/devel/release> ; repo cross-reference (`helix_code/go.mod`, `helix_code/Makefile`, `./helix`, `CLAUDE.md` §3.1/§3.4).